

Internship Report

Submitted By	Suyash Shukla
Project Name	IoT & Robotics Projects (Automated Vehicle, Sentiment Analysis & Smart Home Automation)
Module Name	Internet of Things (IoT) & Robotics
Submission Date	20/03/2026
Institute Name	St. Joseph's University
Mentor Name	Anil Vhatkar
Designation	Industry Mentor

Automated Vehicle Using Arduino, Ultrasonic Sensor and PIR Sensor

1. Introduction

The rapid development of automation and robotics has led to the creation of intelligent systems capable of sensing their environment and making decisions accordingly. This project demonstrates an automated vehicle system using an **Arduino Uno microcontroller**, which controls two DC motors using an **L293D motor driver**. The system uses an **ultrasonic distance sensor** to detect obstacles in front of the vehicle and adjust the speed accordingly. A **PIR motion sensor** is used to detect motion in front of the vehicle and trigger a reverse movement. Additionally, two **push buttons** are used to manually control the direction of the vehicle, allowing it to move left or right.

This project demonstrates the integration of sensors, actuators, and microcontrollers to create a simple intelligent robotic system.

2. Objective

The main objectives of this project are:

- To design an automated vehicle using Arduino.
 - To control two DC motors using an L293D motor driver.
 - To detect obstacles using an ultrasonic sensor.
 - To control speed automatically based on distance.
 - To detect motion using a PIR sensor and trigger backward movement.
 - To control vehicle direction using push buttons.
-

3. Components Required

Component	Quantity
Arduino Uno	1
L293D Motor Driver IC	1
Ultrasonic Sensor (HC-SR04)	1
PIR Motion Sensor	1
DC Motors	2
Push Buttons	2
Breadboard	1
Jumper Wires	Multiple
Power Supply	5V

4. Circuit Description

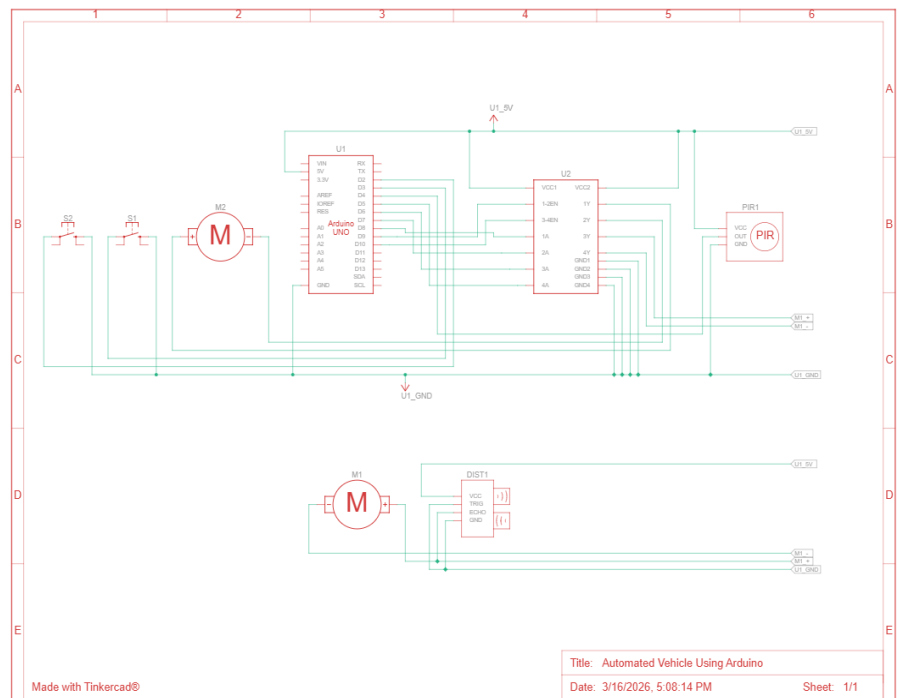
The Arduino Uno acts as the main controller of the system. It receives input signals from the ultrasonic sensor, PIR sensor, and push buttons, then processes these signals to control the DC motors through the L293D motor driver.

The L293D motor driver allows the Arduino to control the direction and speed of two motors. Speed control is achieved using **PWM signals** sent to the enable pins of the motor driver.

The ultrasonic sensor measures the distance to an obstacle by sending ultrasonic waves and calculating the time taken for the echo to return.

The PIR sensor detects motion in front of the vehicle and signals the Arduino to reverse the motors.

Push buttons are used to manually control the direction of the vehicle.



5. Working Principle

Forward Motion

The vehicle moves forward when no obstacle is detected. The ultrasonic sensor continuously measures the distance in front of the vehicle.

Automatic Speed Control

The vehicle adjusts its speed according to the distance from an obstacle.

Distance	Gear	Speed
>300 cm	Gear 5	Maximum speed
250–300 cm	Gear 4	High speed
200–250 cm	Gear 3	Medium speed
150–200 cm	Gear 2	Low speed
100–150 cm	Gear 1	Very slow
<100 cm	Stop	Vehicle stops

Backward Motion

If the PIR sensor detects motion in front of the vehicle, the system immediately stops and moves backward.

Right Direction

If the right push button is pressed, the vehicle turns right by rotating the motors in opposite directions.

Left Direction

If the left push button is pressed, the vehicle turns left by rotating the motors in opposite directions.

6. Arduino Code

```
#define ENA 9
#define IN1 8
#define IN2 7

#define ENB 10
#define IN3 6
#define IN4 5

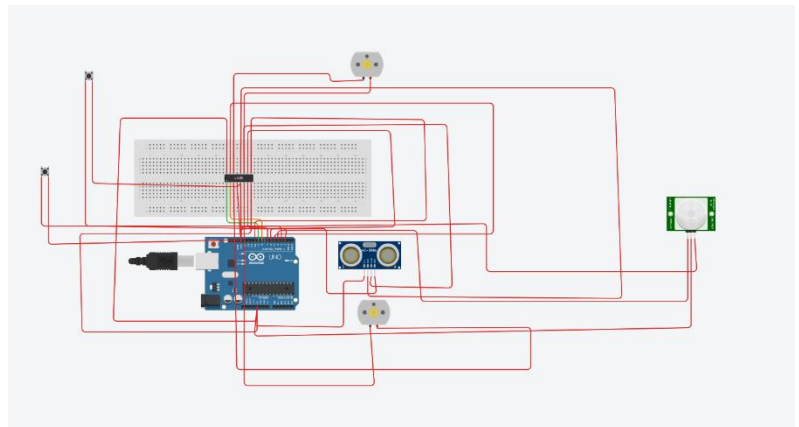
#define trigPin 12
#define echoPin 11

#define pir 4
#define rightBtn 2
#define leftBtn 3

long duration;
int distance;

void setup()
{
  pinMode(ENA,OUTPUT);
  pinMode(IN1,OUTPUT);
  pinMode(IN2,OUTPUT);

  pinMode(ENB,OUTPUT);
  pinMode(IN3,OUTPUT);
  pinMode(IN4,OUTPUT);
```



```
pinMode(trigPin,OUTPUT);  
pinMode(echoPin,INPUT);
```

```
pinMode(pir,INPUT);
```

```
pinMode(rightBtn,INPUT_PULLUP);  
pinMode(leftBtn,INPUT_PULLUP);
```

```
Serial.begin(9600);  
}
```

```
int getDistance()  
{  
    digitalWrite(trigPin,LOW);  
    delayMicroseconds(2);  
  
    digitalWrite(trigPin,HIGH);  
    delayMicroseconds(10);  
    digitalWrite(trigPin,LOW);  
  
    duration=pulseIn(echoPin,HIGH);  
    distance=duration*0.034/2;  
  
    return distance;  
}
```

```
void forward(int speedVal)  
{  
    digitalWrite(IN1,HIGH);  
    digitalWrite(IN2,LOW);  
  
    digitalWrite(IN3,HIGH);  
    digitalWrite(IN4,LOW);  
  
    analogWrite(ENA,speedVal);  
    analogWrite(ENB,speedVal);  
}
```

```
void backward()  
{  
    digitalWrite(IN1,LOW);  
    digitalWrite(IN2,HIGH);
```

```
digitalWrite(IN3,LOW);  
digitalWrite(IN4,HIGH);
```

```
analogWrite(ENA,150);  
analogWrite(ENB,150);  
}
```

```
void stopMotor()  
{  
  analogWrite(ENA,0);  
  analogWrite(ENB,0);  
}
```

```
void right()  
{  
  digitalWrite(IN1,HIGH);  
  digitalWrite(IN2,LOW);  
  
  digitalWrite(IN3,LOW);  
  digitalWrite(IN4,HIGH);  
}
```

```
void left()  
{  
  digitalWrite(IN1,LOW);  
  digitalWrite(IN2,HIGH);  
  
  digitalWrite(IN3,HIGH);  
  digitalWrite(IN4,LOW);  
}
```

```
void loop()  
{  
  distance=getDistance();  
  
  if(digitalRead(pir)==HIGH)  
  {  
    stopMotor();  
    delay(1000);  
    backward();  
    delay(2000);  
  }
```

```
else if(digitalRead(rightBtn)==LOW)
{
    right();
}

else if(digitalRead(leftBtn)==LOW)
{
    left();
}

else
{
    if(distance>300)
        forward(255);

    else if(distance>250)
        forward(200);

    else if(distance>200)
        forward(150);

    else if(distance>150)
        forward(100);

    else if(distance>100)
        forward(60);

    else
        stopMotor();
}

delay(100);
}
```

7. Conclusion

This project demonstrates how sensors and motor drivers can be integrated with a microcontroller to create an automated robotic vehicle. The system successfully adjusts its speed based on obstacle distance, detects motion using a PIR sensor, and allows directional control through push buttons.

Such systems form the foundation for advanced robotics, autonomous vehicles, and intelligent transportation systems.

https://www.tinkercad.com/things/ltT0q0DeupH-automated-vehicle-using-arduino?sharecode=XNlYaY3xpX48scK9ElF_IuV0c8Js7n1a1eV5B1Ivrd0

Sentiment Analysis using ThingSpeak and MIT App Inventor

1. Introduction

Sentiment analysis is a technique used to identify and categorize opinions as positive or negative. In this project, a mobile application is developed using **MIT App Inventor** that allows users to submit positive or negative feedback. The feedback data is stored in the **ThingSpeak IoT cloud platform**, where it is visualized using charts and numerical widgets. A **MATLAB script** is also implemented to monitor the feedback and send an email alert if the number of negative responses exceeds a specified threshold.

2. Objective

The objectives of this project are:

- To design a feedback system using a mobile application.
 - To store and visualize feedback data using the ThingSpeak cloud platform.
 - To analyze feedback using MATLAB.
 - To generate an email alert when negative feedback exceeds a predefined limit.
-

3. Tools and Technologies Used

Tool	Purpose
ThingSpeak	Cloud platform for IoT data storage and visualization
MIT App Inventor	Mobile app development platform
MATLAB Analysis	Data processing and email alert system
Android Smartphone	Testing the application

4. Creating ThingSpeak Channel

1. Login to ThingSpeak.
2. Click **Channels** → **New Channel**.
3. Enter the channel name **Sentiment_Analysis**.
4. Add two fields:

Field	Name
Field 1	Positive Feedback
Field 2	Negative Feedback

5. Save the channel.

5. Adding Widgets and Charts

Widgets and charts were added to visualize feedback data.

Widgets

- Numeric display for **Positive Feedback**
- Numeric display for **Negative Feedback**

Charts

- Chart for **Positive Feedback**
- Chart for **Negative Feedback**

These visualizations help track user sentiment in real time.

6. API Keys

ThingSpeak automatically generates API keys for communication with the channel.

- **Write API Key:** Used to send feedback data to ThingSpeak.
- **Read API Key:** Used to read data from the channel.

These keys are used inside the mobile application to update feedback values.

7. Mobile Application Development (MIT App Inventor)

A mobile application was developed using MIT App Inventor with the following components:

Interface Components

- Button1 – Positive Feedback
- Button2 – Negative Feedback
- WebViewer1 – Display positive feedback chart
- WebViewer2 – Display negative feedback chart

- Clock – Used to create a delay between requests
- Web component – Used to send data to ThingSpeak

Working of Buttons

When the **Positive Feedback button** is clicked, the application sends data to ThingSpeak using the following API request:

[https://api.thingspeak.com/update?api_key= G85D3FC8EA2S3O73=1](https://api.thingspeak.com/update?api_key=G85D3FC8EA2S3O73=1)

When the **Negative Feedback button** is clicked:

[https://api.thingspeak.com/update?api_key= G85D3FC8EA2S3O73=1](https://api.thingspeak.com/update?api_key=G85D3FC8EA2S3O73=1)

8. Delay Mechanism

ThingSpeak free version allows updates only **once every 10 seconds**. To handle this limitation, a **Clock component** is used.

Working:

1. When a button is pressed, both buttons are disabled.
2. Data is sent to ThingSpeak.
3. Clock timer runs for 10 seconds.
4. After 10 seconds, buttons are enabled again.

This prevents sending requests too frequently.

9. Data Visualization in Mobile App

The charts from ThingSpeak are displayed in the mobile application using WebViewer components.

Positive chart URL:

[https://thingspeak.com/channels/ 3302036/charts/1](https://thingspeak.com/channels/3302036/charts/1)

Negative chart URL:

<https://thingspeak.com/channels/3302036/charts/2>

These charts display real-time feedback data inside the app.

10. MATLAB Email Alert System

A MATLAB script is used to monitor negative feedback and trigger an email alert.

MATLAB Code

```
readChannelID = 3302036;
fieldID = 2;

data = thingSpeakRead(readChannelID,'Fields',fieldID,'NumPoints',10);

negCount = sum(data);

if negCount > 5
    sendmail('suyash.shukla911@gmail.com','Alert','Negative feedback exceeded 5');
end
```

Working

- The script reads recent data from the ThingSpeak channel.
 - It calculates the number of negative feedback entries.
 - If the count exceeds 5, an email alert is sent to the specified email address.
-

11. Application Testing

The application was tested on an Android device using the MIT App Inventor Companion app.

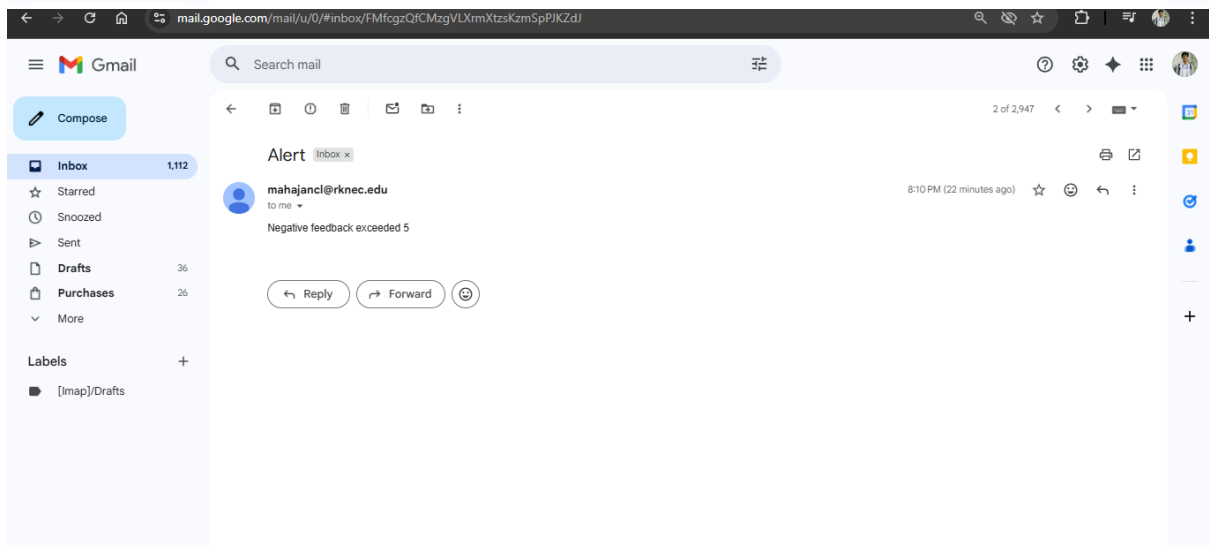
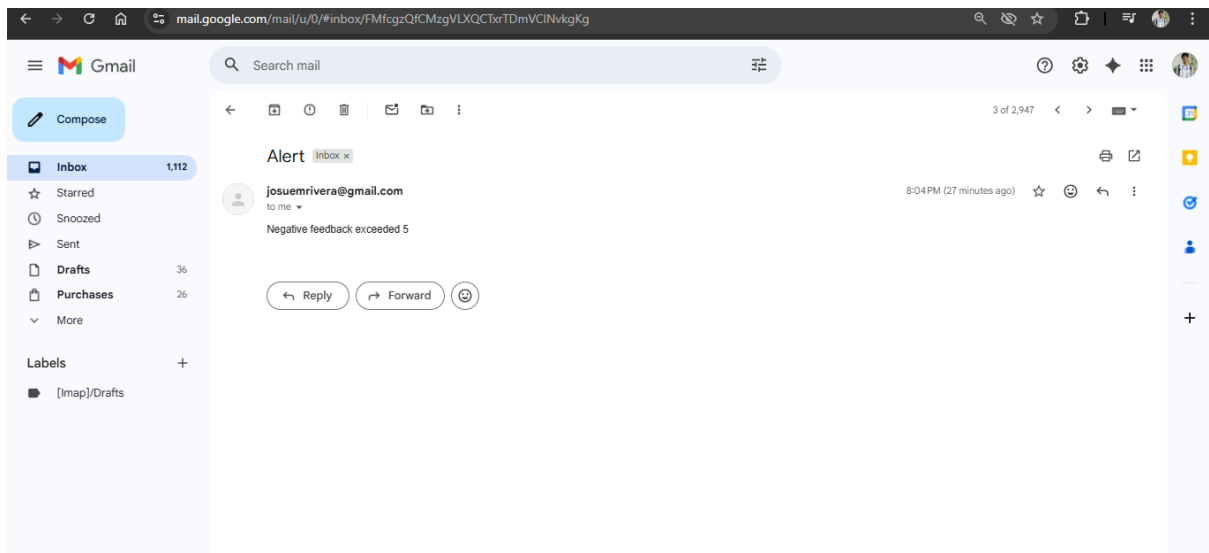
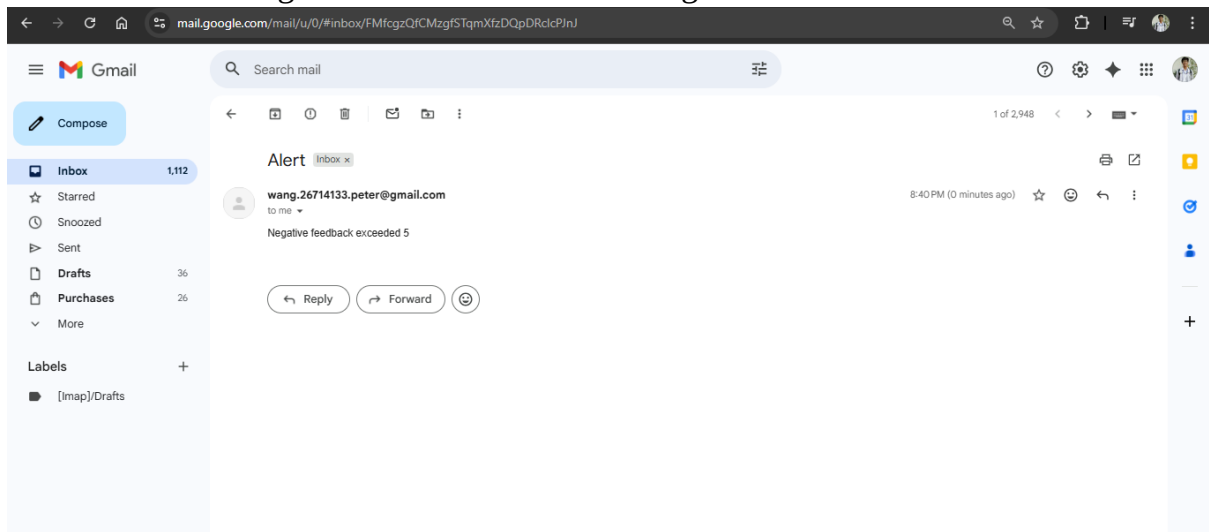
Test results:

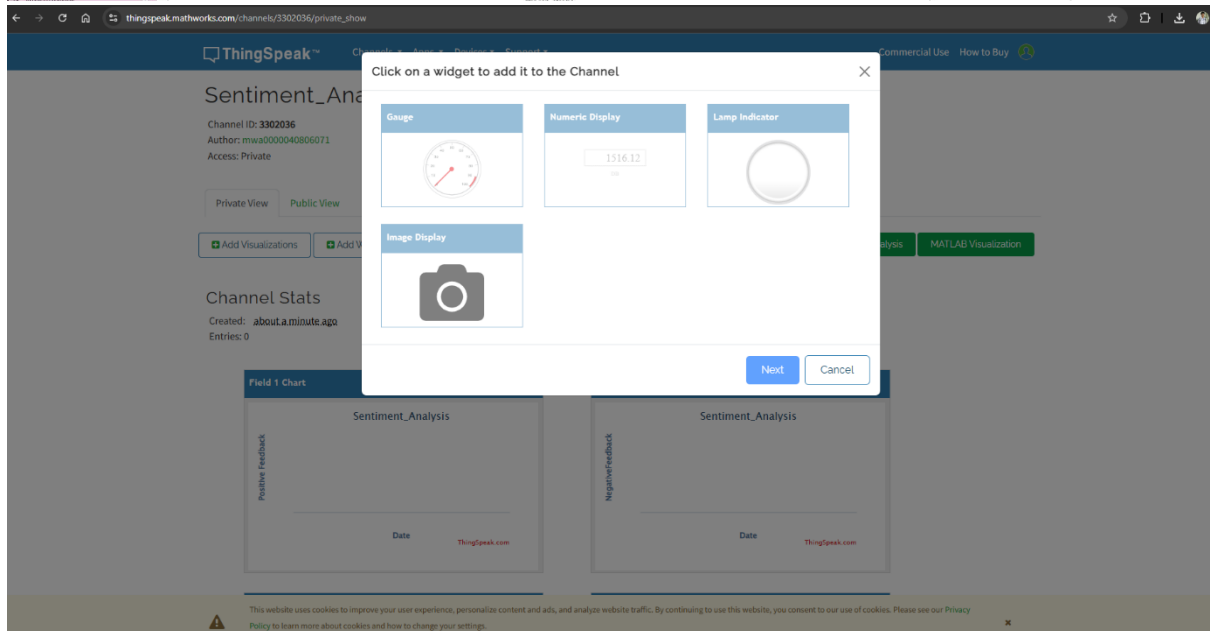
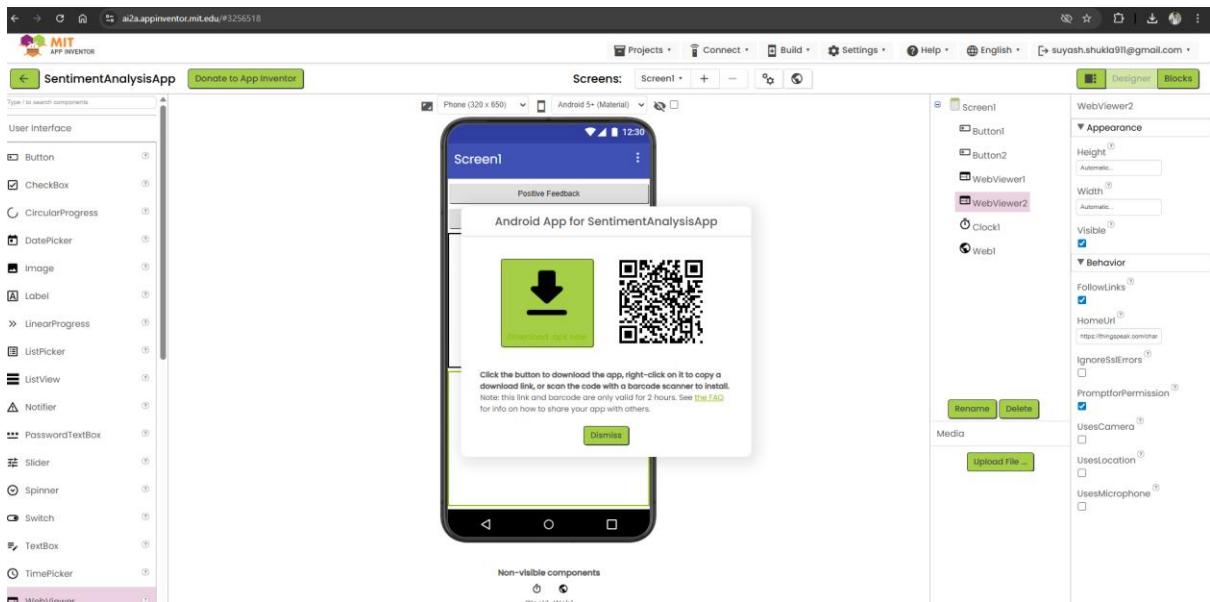
- Positive feedback button successfully updates Field1.
 - Negative feedback button successfully updates Field2.
 - Charts update automatically with new data.
 - MATLAB script runs successfully for email alerts.
-

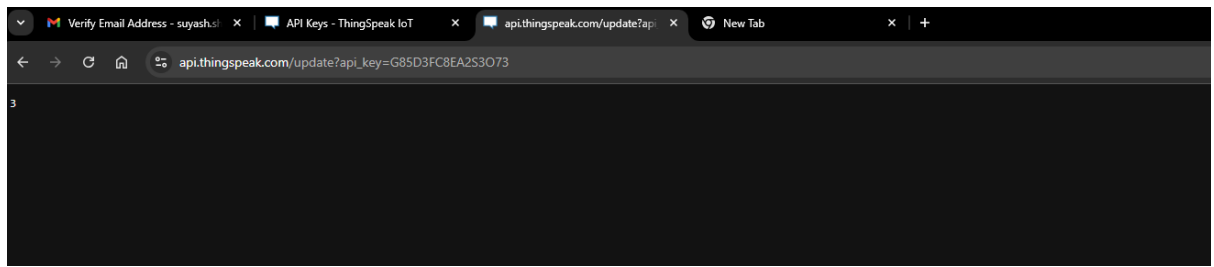
12. Conclusion

This project demonstrates how mobile applications and cloud platforms can be integrated to collect and analyze user feedback in real time. By combining MIT App Inventor and ThingSpeak, a simple sentiment analysis system was developed. The system allows users to submit feedback through a mobile interface while enabling

real-time monitoring and automated alerts for negative feedback.







6:25

🕒 📶 VoLTE 📶 🔋 92%

Screen1



Positive Feedback

Negative FeedBack

Sentiment_Analysis



18:00 18:05 18:10 18:15 18:20

Date

ThingSpeak.com

Sentiment_Analysis



18:00 18:05 18:10 18:15 18:20

Date

ThingSpeak.com

Sentiment_Analysis

Channel ID: 3302036
Author: mwa000040806071
Access: Private

Private View Public View Channel Settings Sharing API Keys Data Import / Export

Write API Key

Key

[Generate New Write API Key](#)

Read API Keys

Key

Note

[Save Note](#) [Delete API Key](#)

[Add New Read API Key](#)

Help

API keys enable you to write data to a channel or read data from a private channel. API keys are auto-generated when you create a new channel.

API Keys Settings

- **Write API Key:** Use this key to write data to a channel. If you feel your key has been compromised, click **Generate New Write API Key**.
- **Read API Keys:** Use this key to allow other people to view your private channel feeds and charts. Click **Generate New Read API Key** to generate an additional read key for the channel.
- **Note:** Use this field to enter information about channel read keys. For example, add notes to keep track of users with access to your channel.

API Requests

Write a Channel Feed

```
GET https://api.thingspeak.com/update?api_key=G85D3FC8EA253073&field1=0
```

Read a Channel Feed

```
GET https://api.thingspeak.com/channels/3302036/feeds.json?api_key=EXNGIV9I1ZNN66HI
```

Read a Channel Field

```
GET https://api.thingspeak.com/channels/3302036/fields/1.json?api_key=EXNGIV9I1ZNN66HI
```

This website uses cookies to improve your user experience, personalize content and ads, and analyze website traffic. By continuing to use this website, you consent to our use of cookies. Please see our [Privacy](#)

Projects Connect Build Settings Help English

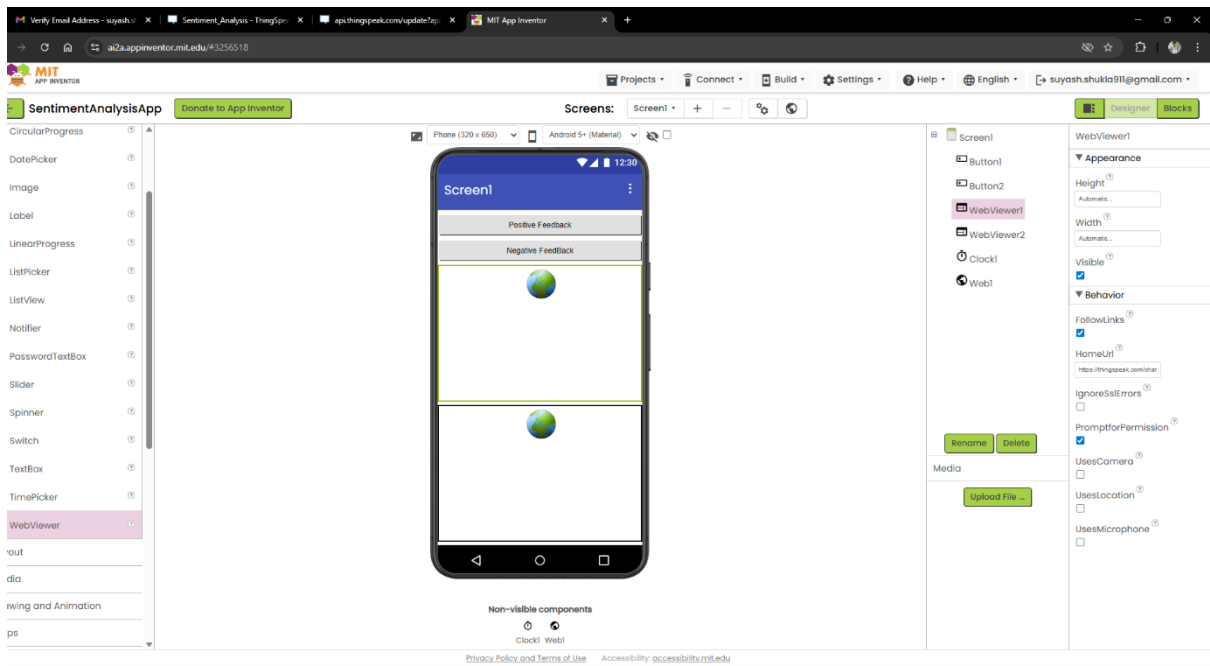
sisApp [Donate to App Inventor](#)

Screens: Screen1 + - ⚙️ ↻

```
when Button1 Click
do
  set Button1 Enabled to false
  set Button2 Enabled to false
  call WebViewer1 GoToUrl
  url https://api.thingspeak.com/update?api_key=G85D...
  set Clock1 TimerEnabled to true
```

```
when Button2 Click
do
  set Button1 Enabled to false
  set Button2 Enabled to false
  call WebViewer1 GoToUrl
  url https://api.thingspeak.com/update?api_key=G85D...
  set Clock1 TimerEnabled to true
```

```
when Clock1 Timer
do
  set Button1 Enabled to true
  set Button2 Enabled to true
  set Clock1 TimerEnabled to false
```



Templates:

- ☒ Custom (no starter code)
- ☐ Get data from a private channel
- ☐ Get data from a public channel
- ☐ Get data from a webpage

Examples: Sample code to analyze and transform data

- ☐ Calculate and display average humidity
- ☐ Calculate wind chill and update channel
- ☐ Remove outliers from wind speed data
- ☐ Convert temperature units
- ☐ Calculate high and low temperatures
- ☐ Read Channel to Trigger Email
- ☐ Replace missing values in data
- ☐ Analyze text for the most common color
- ☐ Read live web data for vessels at the port of Boston
- ☐ Scrape web temperature data

Create

Help

Templates

MATLAB Analysis templates provide sample MATLAB code for analyzing data and writing it to a ThingSpeak channel. If you are new to MATLAB, you can learn interactively at [MATLAB Academy](#).

Examples

To see MATLAB Analysis in action, select the example and click **Create**.

These examples read data from public ThingSpeak channels:

- [Calculate and display average humidity](#) over the last 60 minutes, and write the results to a new channel.
- [Calculate wind chill and update channel](#) by writing to a new channel at regular intervals.
- [Remove outliers from wind speed data](#) over the past six hours, and write data to a new channel.
- [Convert temperature units](#) from Fahrenheit to Celsius, and write the results to a new channel.
- [Calculate high and low temperatures](#) over the past 24 hours, and write the data with timestamps to a new channel.
- [Trigger Email](#) by analyzing daily soil moisture values.
- [Replace missing values in data](#) of a weather channel, and clean the data using a list-wise deletion algorithm. Then display the missing values, or write data to a new channel.
- [Analyze text for the most common color](#) requested on the public Cheerlights channel, and write the data to a new channel.

These examples scrape data from websites:

- [Read live web data for vessels at the port of Boston](#) from the MarineTraffic website, count the number of vessels, and write the data to a new channel.
- [Scrape web temperature data](#) from the National Weather Service website, and write the data to a new channel.

New to MATLAB?

Help

Documentation

Read Channel to Trigger Email 1

```
1 readChannelID = 3302036;
2 fieldID = 2;
3
4 data = thingsSpeakRead(readChannelID,'Fields',fieldID,'NumPoints',10);
5
6 negCount = sum(data);
7
8 if negCount > 5
9     sendmail('suyash.shukla911@gmail.com','Alert','Negative feedback exceeded 5');
10 end
```

Save*

New Channel

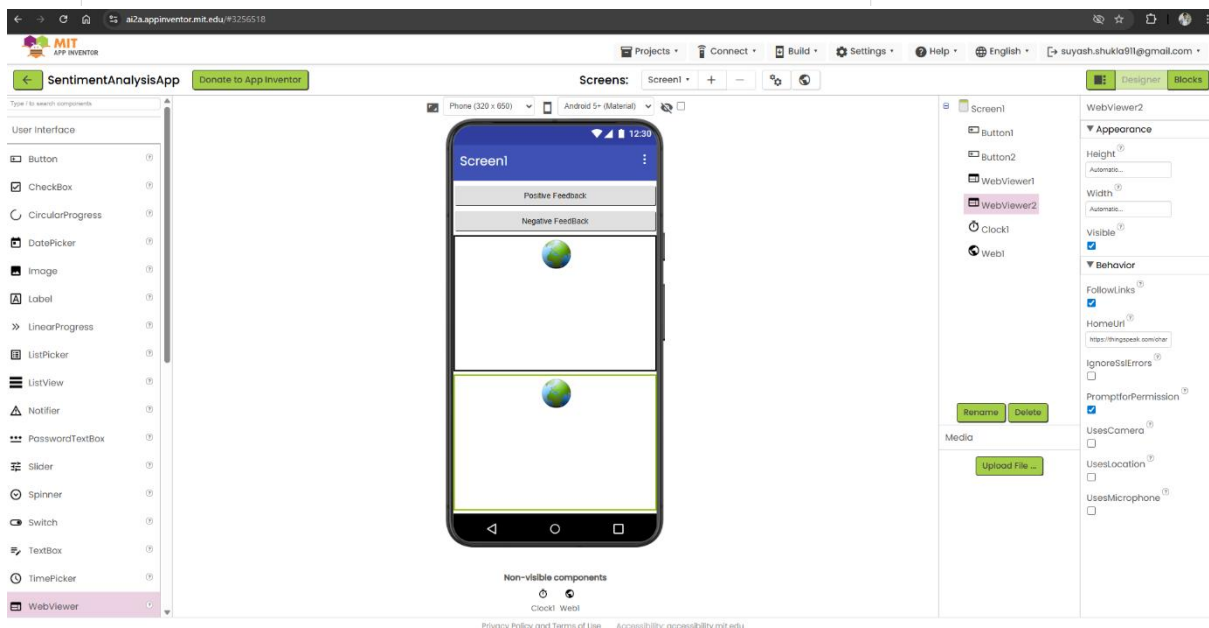
Name: **Sentiment_Analysis**
Channel ID: 3302036
Access: Public
Read API Key: **EXNGIV9I1ZNN66HI**
Write API Key: **G85D3FC8EA2S3073**
Fields:
1: Positive Feedback
2: NegativeFeedback

➤ Analyze Data

- thingSpeakRead

- thingSpeakWrite

- `urlfilter`



SMART HOME AUTOMATION SYSTEM USING I2C

(IoT & Robotics Major Project)

1. Introduction

Smart home automation systems are designed to improve convenience, security, and energy efficiency by automatically controlling household devices.

In this project, a **Smart Home Automation System** is developed using **two Arduino UNO boards communicating via I2C protocol**. The system integrates multiple sensors and actuators to perform intelligent operations such as security monitoring, climate control, lighting automation, and energy tracking.

2. Objective

The main objectives of this project are:

- To design a smart home system using Arduino
 - To implement **I2C communication between two microcontrollers**
 - To provide secure access using keypad-based password
 - To automate lighting and temperature control
 - To implement motion detection and alarm system
 - To simulate solar tracking system
-

3. Components Used

Component	Quantity	Purpose
Arduino UNO	2	Master & Slave controllers
Ultrasonic Sensor	1	System activation
Keypad (4x4)	1	Password input
Servo Motor	2	Door & solar tracking
PIR Sensor	1	Motion detection
TMP36 Sensor	1	Temperature sensing
LDR	1	Light sensing

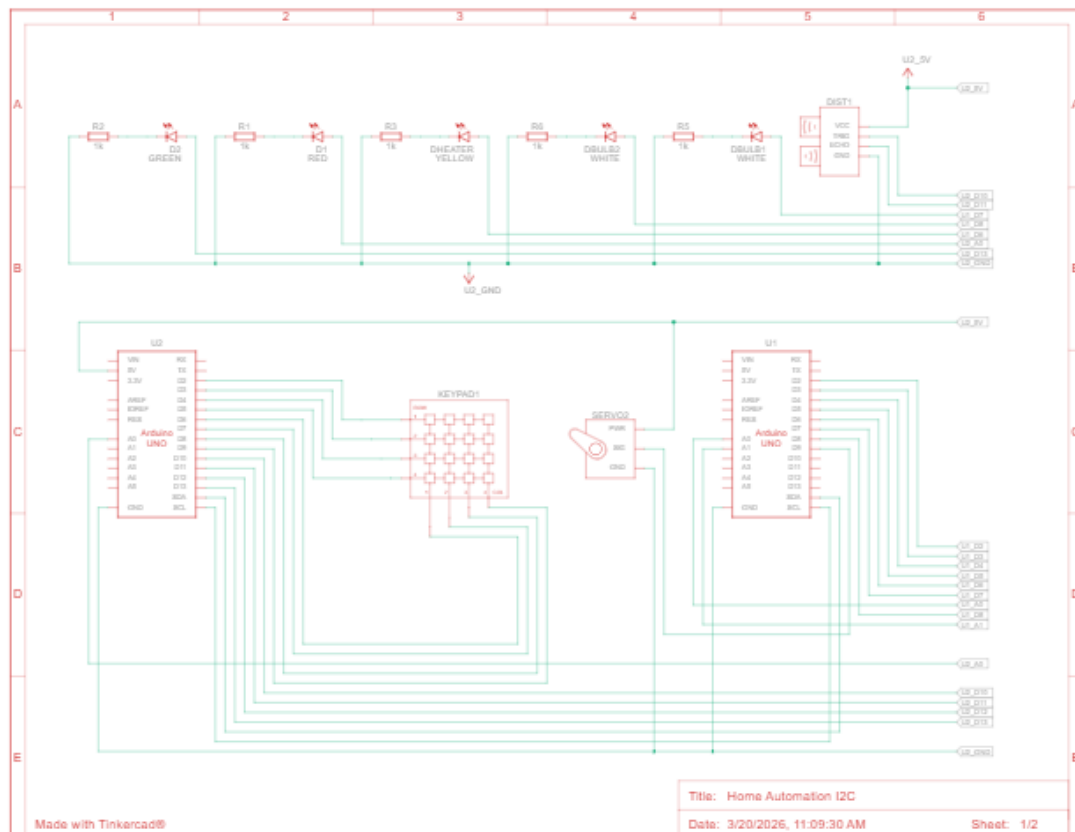
LEDs	Multiple	Lights, heater, fan indicators
Buzzer	1	Alarm system
Resistors	Multiple	Current limiting
Breadboard	1	Circuit connections
Battery (9V)	1	Power supply

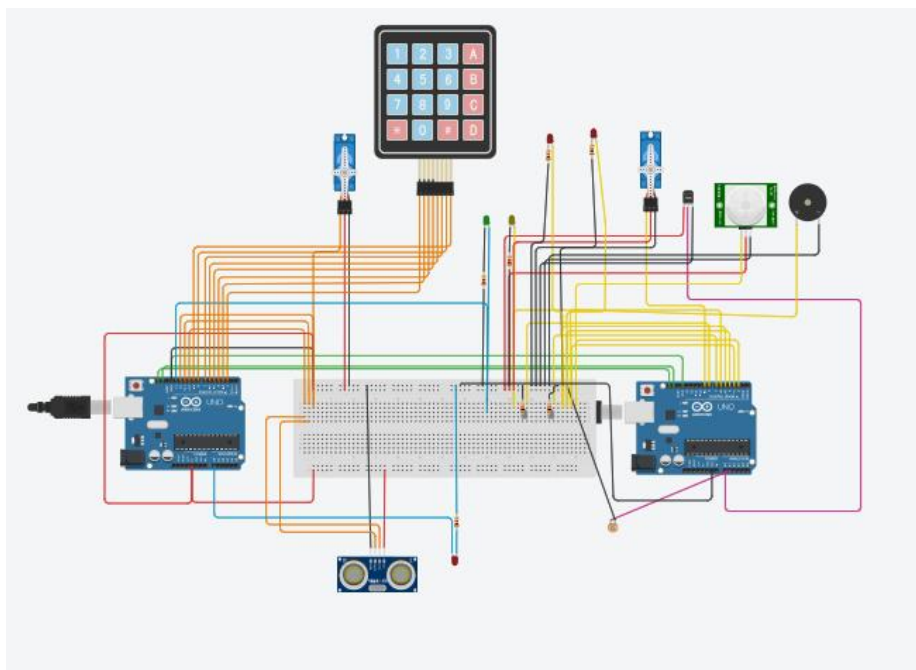
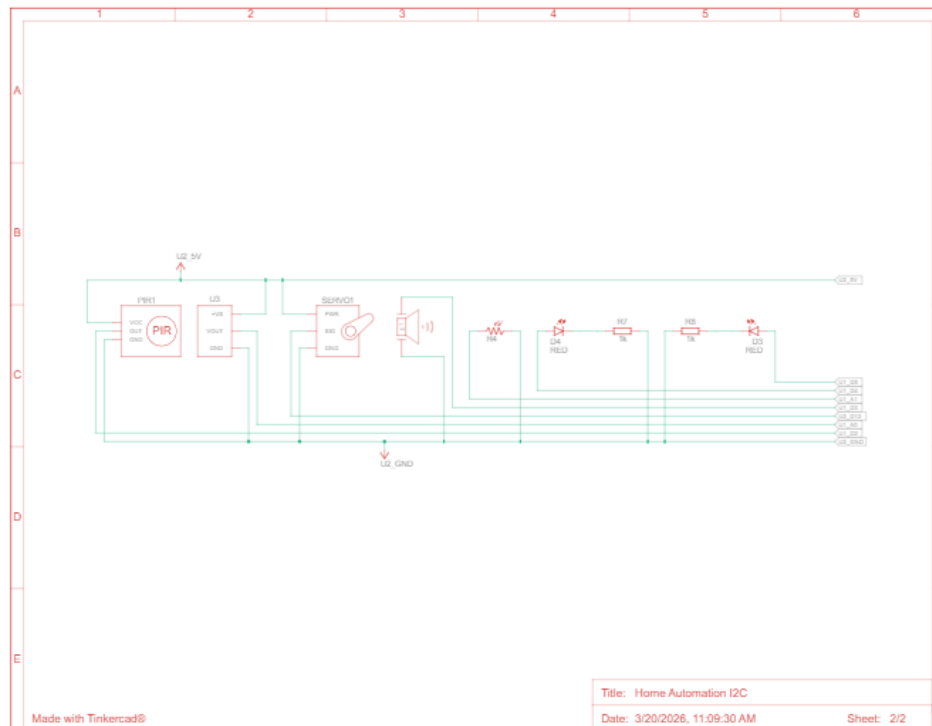
4. System Architecture

The system is divided into two parts:

● Arduino 1 (Master)

- Ultrasonic sensor (activation)
- Keypad (password input)
- Door control (servo motor)
- Status LEDs





● Arduino 2 (Slave)

- Temperature sensor (fans/heater control)
- LDR (lighting + solar tracking)
- PIR sensor (security system)
- Buzzer (alarm)

Both Arduino boards communicate using **I2C protocol (SDA & SCL lines)**

5. Working Principle

5.1 System Activation

- Initially, the system is OFF
- Ultrasonic sensor detects distance
- If distance < 50 cm $\rightarrow$ system activates

5.2 Door Access System

- User enters password using keypad
- Password ends with '#'
- If correct $\rightarrow$ door opens using servo motor
- Green LED ON (open), Red LED OFF
- If wrong 3 times $\rightarrow$ alarm triggered

5.3 Climate Control

- Temperature is measured using TMP36
- If temperature $> 27^{\circ}\text{C}$ $\rightarrow$ Fans ON
- If temperature $< 17^{\circ}\text{C}$ $\rightarrow$ Heater ON

5.4 Light Control

- LDR detects light intensity
- If light $<$ threshold $\rightarrow$ Lights ON
- Otherwise $\rightarrow$ Lights OFF

5.5 Solar Tracking System

- LDR value changes with light intensity
 - Servo motor rotates from 0° to 180°
 - Simulates solar panel movement
-

5.6 Security System

- PIR sensor detects motion
 - If motion detected → buzzer sounds
 - Provides safety for restricted areas
-

6. Circuit Diagram

The circuit consists of two Arduino boards connected using I2C communication lines (SDA and SCL).

The detailed circuit diagram is shown in the project design

7. Arduino Code

```
#include <Wire.h>

#include <Servo.h>

#include <Keypad.h>

// ===== KEYPAD =====

const byte ROWS = 4;
const byte COLS = 4;

char keys[ROWS][COLS] = {
  {'1','2','3','A'},
  {'4','5','6','B'},
  {'7','8','9','C'},
  {'*','0','#','D'}
};

byte rowPins[ROWS] = {2,3,4,5};
byte colPins[COLS] = {6,7,8,9};
```

```
Keypad keypad = Keypad(makeKeymap(keys), rowPins, colPins, ROWS, COLS);
```

```
#define trigPin 10
```

```
#define echoPin 11
```

```
Servo door;
```

```
bool systemActive = false;
```

```
String password = "1234#";
```

```
String input = "";
```

```
int attempts = 0;
```

```
void setup()
```

```
{
```

```
  Wire.begin(); // MASTER
```

```
  Serial.begin(9600);
```

```
  pinMode(trigPin, OUTPUT);
```

```
  pinMode(echoPin, INPUT);
```

```
  door.attach(12);
```

```
}
```

```
void loop()
```

```
{
```

```
  ultrasonicActivation();
```

```
if(systemActive)
{
    doorAccess();

    // send signal to slave
    Wire.beginTransmission(8);
    Wire.write(1); // system ON
    Wire.endTransmission();
}
else
{
    Wire.beginTransmission(8);
    Wire.write(0); // system OFF
    Wire.endTransmission();
}
}
```

// ♦ Ultrasonic

```
void ultrasonicActivation()
{
    long duration;
    int distance;

    digitalWrite(trigPin, LOW);
    delayMicroseconds(2);

    digitalWrite(trigPin, HIGH);
    delayMicroseconds(10);
    digitalWrite(trigPin, LOW);
```

```
duration = pulseIn(echoPin, HIGH);
```

```
distance = duration * 0.034 / 2;
```

```
if(distance < 50)
```

```
{
```

```
    systemActive = true;
```

```
}
```

```
}
```

```
// ♦ Door
```

```
void doorAccess()
```

```
{
```

```
    char key = keypad.getKey();
```

```
    if(key)
```

```
    {
```

```
        input += key;
```

```
        Serial.println(input);
```

```
    }
```

```
    if(key == '#')
```

```
    {
```

```
        if(input == password)
```

```
        {
```

```
            Serial.println("Access Granted");
```

```
            door.write(90);
```

```
            delay(3000);
```

```
    door.write(0);  
}  
else  
{  
    Serial.println("Wrong Password");  
}  
  
input = "";  
}  
}
```

8. Results

The system was successfully implemented and tested in Tinkercad simulation.

- System activates based on proximity
 - Door opens using keypad password
 - Temperature controls fan and heater
 - Lights operate automatically using LDR
 - Solar tracking works using servo motor
 - Security alarm activates on motion detection
-

9. Advantages

- Improved home security
 - Energy efficient system
 - Modular design using I2C
 - Cost-effective implementation
 - Easy to expand
-

10. Limitations

- Simulation-based system
 - Limited power handling (no real motors)
 - Requires proper wiring for stability
-

```
#define trigPin 10
#define echoPin 11

Servo door;

bool systemActive = false;

String password = "1234#";
String input = "";
int attempts = 0;

void setup()
{
  Wire.begin(); // MASTER

  Serial.begin(9600);

  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);

  door.attach(12);
}

void loop()
{
  ultrasonicActivation();
```

11. Applications

- Smart homes
 - Industrial automation
 - Security systems
 - Energy management systems
-

12. Conclusion

This project successfully demonstrates a smart home automation system using Arduino and I2C communication. The integration of multiple sensors and modules enables efficient control of home appliances and security systems. The system provides a strong foundation for developing advanced IoT-based automation solutions.

Tinkercad link - https://www.tinkercad.com/things/5o9oir6l4JB-home-automation-i2c?sharecode=pqP_679tFe12xoJmHku_E3vjralr4WL0KozR9v4aqT4